

# BACKEND DEVELOPER

## Interview Question Bank

Node.js · Express.js · MongoDB

---

|                 |               |                 |
|-----------------|---------------|-----------------|
| 55 Questions    | 5 Sections    | 3 Levels        |
| Theory + Coding | System Design | Interview-Ready |

Prepared for Senior & Mid-Level Backend Assessments  
Includes Expected Answers · Coding Tasks · System Design

## ■ Table of Contents

---

|                             |           |              |
|-----------------------------|-----------|--------------|
| ■ Beginner Questions        | Q1 – Q12  | 12 questions |
| ■ Intermediate Questions    | Q13 – Q27 | 15 questions |
| ■ Advanced Questions        | Q28 – Q42 | 15 questions |
| ■ Coding / Practical Tasks  | Q43 – Q48 | 6 tasks      |
| ■ ■ System Design Questions | Q49 – Q55 | 7 questions  |

---

## ■ BEGINNER QUESTIONS

---

Foundational concepts — Node.js basics, Express setup, MongoDB intro, REST.

### Q1.

**What is Node.js and how does it differ from browser-based JavaScript?**

#### ■ *Node.js Core*

##### ◆ **Key Points:**

- Node.js is a server-side runtime built on Chrome's V8 engine
  - No DOM/window APIs; provides fs, http, os modules instead
  - Single-threaded, non-blocking I/O model
  - Uses CommonJS (require) and ESM (import) modules
- 

### Q2.

**Explain the Node.js Event Loop in simple terms.**

#### ■ *Event Loop*

##### ◆ **Key Points:**

- Single thread processes one task at a time via a loop
  - Phases: timers → pending callbacks → idle → poll → check → close
  - Offloads I/O to libuv thread pool, callbacks queued when done
  - Microtasks (Promises, process.nextTick) run between phases
- 

### Q3.

**What is the difference between require() and import in Node.js?**

#### ■ *Modules*

##### ◆ **Key Points:**

- require() = CommonJS (synchronous, dynamic); import = ESM (static, async-capable)
  - ESM supports tree-shaking; CommonJS does not
  - Use .mjs or 'type':'module' in package.json for ESM
  - Mixing both in one project needs careful config
- 

### Q4.

**What are callbacks and why can they lead to 'Callback Hell'?**

#### ■ *Async Patterns*

##### ◆ **Key Points:**

- A callback is a function passed as argument, called after async op completes
  - Nesting multiple async callbacks creates deeply indented, hard-to-read code
  - Solved by Promises (.then/.catch) or async/await
  - Error handling becomes inconsistent in deeply nested callbacks
-

## Q5.

### How does async/await work under the hood?

#### ■ *Async/Await*

##### ◆ Key Points:

- async functions return a Promise automatically
- await pauses execution inside the function, not the event loop
- Syntactic sugar over Promise chains
- Always wrap in try/catch for error handling

```
async function fetchUser(id) {
  try {
    const user = await User.findById(id);
    return user;
  } catch (err) {
    throw new Error('User not found');
  }
}
```

---

## Q6.

### What is Express.js and why do we use it?

#### ■ *Express.js*

##### ◆ Key Points:

- Minimal, unopinionated Node.js web framework
  - Provides routing, middleware pipeline, request/response helpers
  - Faster to build APIs than raw http module
  - Large ecosystem of compatible middleware (cors, helmet, morgan)
- 

## Q7.

### Explain Express middleware with an example.

#### ■ *Express Middleware*

##### ◆ Key Points:

- Functions with signature (req, res, next) that execute in a pipeline
- Can modify req/res, end the request cycle, or call next()
- Types: application-level, router-level, error-handling, built-in, third-party
- Order of app.use() calls determines execution order

```
app.use((req, res, next) => {
  console.log(`${req.method} ${req.path}`);
  next(); // pass to next middleware
});
```

---

## Q8.

### What is REST and what are the key HTTP methods?

#### ■ *REST APIs*

##### ◆ Key Points:

- REST = Representational State Transfer, stateless client-server architecture
  - GET (read), POST (create), PUT/PATCH (update), DELETE (remove)
  - Resources identified by URLs; responses use HTTP status codes
  - Stateless: each request must carry all info needed
- 

## Q9.

### What is MongoDB and how does it differ from SQL databases?

#### ■ *MongoDB Basics*

##### ◆ Key Points:

- Document-oriented NoSQL database; stores JSON-like BSON documents
  - Schema-less (flexible structure) vs rigid table schemas in SQL
  - No JOINS natively; data often embedded in documents
  - Horizontally scalable; excellent for hierarchical/varied data
- 

## Q10.

### What is Mongoose and why is it used with MongoDB?

#### ■ *Mongoose*

##### ◆ Key Points:

- ODM (Object Data Modelling) library for MongoDB + Node.js
  - Provides schema validation, type casting, query building
  - Supports middleware (pre/post hooks), virtuals, plugins
  - Makes MongoDB feel more structured and type-safe
- 

## Q11.

### How do you handle environment variables in Node.js?

#### ■ *Config & Security*

##### ◆ Key Points:

- Store secrets in .env file, never commit to version control
  - Use dotenv package: `require('dotenv').config()`
  - Access via `process.env.VARIABLE_NAME`
  - Use separate .env files per environment (dev, staging, prod)
-

## Q12.

### What is the difference between PUT and PATCH?

#### ■ REST APIs

#### ◆ Key Points:

- PUT replaces the entire resource with the request body
  - PATCH applies partial modifications to a resource
  - PUT is idempotent; PATCH can be but isn't required to be
  - PATCH is preferred when updating only specific fields
-

---

## ■ INTERMEDIATE QUESTIONS

---

Real-world patterns — auth, middleware, aggregation, security, project structure.

### Q13.

**Explain Node.js Streams and when you'd use them.**

#### ■ Streams

##### ◆ Key Points:

- Streams process data in chunks rather than loading it all into memory
- 4 types: Readable, Writable, Duplex, Transform
- Ideal for file uploads, video streaming, large CSV processing
- Pipe chains: `readable.pipe(transform).pipe(writable)`

```
const fs = require('fs');
const readStream = fs.createReadStream('big.csv');
const writeStream = fs.createWriteStream('out.csv');
readStream.pipe(writeStream);
```

---

### Q14.

**How does JWT authentication work in Express?**

#### ■ Authentication

##### ◆ Key Points:

- JWT = Header.Payload.Signature encoded in Base64
- Server signs token with secret; client stores it (localStorage/cookie)
- Each request sends token in Authorization: Bearer header
- Server verifies signature without DB lookup — stateless auth
- Use short expiry + refresh tokens for security

```
const token = jwt.sign({ userId: user._id }, process.env.JWT_SECRET, { expiresIn: '15m' });
// Middleware
const auth = (req, res, next) => {
  const token = req.headers.authorization?.split(' ')[1];
  const decoded = jwt.verify(token, process.env.JWT_SECRET);
  req.user = decoded; next();
};
```

---

### Q15.

**What is Express Router and how does it help organise code?**

#### ■ Express Routing

##### ◆ Key Points:

- Router is a mini Express app for grouping related routes
  - Separates concerns: `userRouter`, `productRouter`, `authRouter`
  - Mount routers on prefixes: `app.use('/api/users', userRouter)`
  - Supports middleware scoped to specific route groups
-

## Q16.

### How do you implement global error handling in Express?

#### ■ Error Handling

##### ◆ Key Points:

- Define error-handling middleware with 4 params: (err, req, res, next)
- Must be registered AFTER all routes
- Pass errors using next(err) from any middleware or route
- Distinguish operational errors (4xx) from programmer errors (5xx)

```
app.use((err, req, res, next) => {  
  const status = err.status || 500;  
  res.status(status).json({  
    success: false,  
    message: err.message || 'Internal Server Error'  
  });  
});
```

---

## Q17.

### Explain MongoDB Aggregation Pipeline.

#### ■ MongoDB Aggregation

##### ◆ Key Points:

- Series of stages that transform documents: \$match, \$group, \$sort, \$project, \$lookup
- \$match filters documents (like WHERE in SQL)
- \$group groups and computes values (like GROUP BY)
- \$lookup performs left outer join with another collection

```
db.orders.aggregate([  
  { $match: { status: 'delivered' } },  
  { $group: { _id: 'userId', total: { $sum: '$amount' } } },  
  { $sort: { total: -1 } },  
  { $limit: 10 }  
]);
```

---

## Q18.

### What are MongoDB indexes and when should you use them?

#### ■ MongoDB Indexing

##### ◆ Key Points:

- Indexes store a small sorted portion of data for fast lookups
  - Default \_id index on every collection
  - Single field, compound, text, geospatial, TTL indexes available
  - Over-indexing slows writes; index only frequently-queried fields
  - Use explain() to analyse query performance
-



## Q19.

### How do you prevent SQL/NoSQL injection in Node.js APIs?

#### ■ Security

##### ◆ Key Points:

- MongoDB: avoid passing raw user input as query operators
- Sanitise inputs with express-mongo-sanitize package
- Validate all inputs with Joi or Zod before touching DB
- For SQL: always use parameterised queries / prepared statements

```
// Dangerous:
User.find({ username: req.body.username });

// Safe:
const { error, value } = schema.validate(req.body);
if (error) return res.status(400).json({ error });
```

---

## Q20.

### Explain CORS and how to configure it in Express.

#### ■ Security & HTTP

##### ◆ Key Points:

- Cross-Origin Resource Sharing — browser security policy for cross-domain requests
- Server sends Access-Control-Allow-Origin headers to permit domains
- Use the cors npm package for easy configuration
- Configure allowed origins, methods, headers per environment

```
const cors = require('cors');
app.use(cors({
  origin: process.env.CLIENT_URL,
  methods: ['GET', 'POST', 'PUT', 'DELETE'],
  credentials: true
}));
```

---

## Q21.

### What is the difference between process.nextTick() and setImmediate()?

#### ■ Event Loop

##### ◆ Key Points:

- process.nextTick() fires before the next event loop iteration (microtask queue)
  - setImmediate() fires in the check phase of the next event loop iteration
  - nextTick has higher priority; can starve I/O if overused
  - Use setImmediate() for deferred work that shouldn't block I/O
-

## Q22.

### How do you implement pagination in a MongoDB + Express API?

#### ■ API Design

##### ◆ Key Points:

- Cursor-based or offset/limit pagination
- Offset: `skip((page-1)*limit).limit(limit)` — simple but slow on large datasets
- Cursor-based: query after last seen `_id` — efficient, consistent
- Always include total count and `hasNextPage` in response

```
const page = parseInt(req.query.page) || 1;
const limit = parseInt(req.query.limit) || 10;
const users = await User
  .find({})
  .skip((page - 1) * limit)
  .limit(limit)
  .lean();
```

---

## Q23.

### What are Mongoose middleware (pre/post hooks)?

#### ■ Mongoose

##### ◆ Key Points:

- Functions that run before (pre) or after (post) certain operations
- Common uses: hash password before save, log after find, cascade delete
- Document middleware: save, validate, remove, init
- Query middleware: find, findOne, updateOne, deleteOne

```
userSchema.pre('save', async function(next) {
  if (this.isModified('password')) {
    this.password = await bcrypt.hash(this.password, 12);
  }
  next();
});
```

---

## Q24.

### How do you structure a production Express.js project?

#### ■ Project Architecture

##### ◆ Key Points:

- Layered architecture: Routes → Controllers → Services → Models
  - Separate config, middleware, utils into own directories
  - Never put business logic in route handlers
  - Use `index.js` barrel exports for clean imports
-

## Q25.

### Explain rate limiting and why it's important.

#### ■ Security

##### ◆ Key Points:

- Limits number of requests a client can make in a time window
- Protects against brute-force attacks, DDoS, API abuse
- Use express-rate-limit package
- Apply stricter limits on auth endpoints

```
const rateLimit = require('express-rate-limit');
app.use('/api/auth', rateLimit({
  windowMs: 15 * 60 * 1000, // 15 min
  max: 10, // 10 requests per window
  message: 'Too many requests'
}));
```

---

## Q26.

### What is connection pooling in MongoDB and why does it matter?

#### ■ MongoDB Performance

##### ◆ Key Points:

- Pool of reusable DB connections instead of creating new ones per request
  - Mongoose manages a pool automatically (default size: 5)
  - Set poolSize option based on expected concurrent load
  - Prevents connection exhaustion under high traffic
- 

## Q27.

### How does bcrypt password hashing work?

#### ■ Security

##### ◆ Key Points:

- bcrypt applies salt + multiple hash rounds (cost factor)
  - Salt prevents rainbow table attacks
  - Higher cost factor = slower hash = harder to brute-force
  - Never store plain-text or MD5/SHA1 hashed passwords
  - Use bcrypt.compare() for verification — never decrypt
-

---

## ■ ADVANCED QUESTIONS

---

Production depth — performance, transactions, scaling, security hardening.

### Q28.

**Explain the Worker Threads module and when to use it.**

■ *Node.js Internals*

#### ◆ Key Points:

- Worker Threads enable true multi-threading in Node.js
- Main thread and workers share memory via SharedArrayBuffer
- Use for CPU-intensive tasks: image processing, encryption, ML inference
- Not needed for I/O-bound tasks — the event loop handles those efficiently

```
const { Worker } = require('worker_threads');
const worker = new Worker('./heavy-task.js', { workerData: { input } });
worker.on('message', result => console.log(result));
```

---

### Q29.

**How would you implement a robust refresh token rotation system?**

■ *Authentication & Security*

#### ◆ Key Points:

- Access token (short-lived 15m) + Refresh token (long-lived 7d)
  - Store hashed refresh tokens in DB with userId and expiry
  - On refresh: validate token, issue new pair, invalidate old token (rotation)
  - On logout: delete all refresh tokens for user
  - Detect reuse attacks: if old token used, revoke ALL user sessions
- 

### Q30.

**What are MongoDB transactions and when are they needed?**

■ *MongoDB Transactions*

#### ◆ Key Points:

- Multi-document ACID transactions available since MongoDB 4.0
- Use when multiple documents must update atomically (e.g. bank transfer)
- Requires replica set or sharded cluster
- Increased write latency — use only when true atomicity is needed

```
const session = await mongoose.startSession();
session.startTransaction();
try {
  await Account.updateOne({ _id: from }, { $inc: { balance: -amount } }, { session });
  await Account.updateOne({ _id: to }, { $inc: { balance: amount } }, { session });
  await session.commitTransaction();
} catch (e) {
  await session.abortTransaction();
}
```

---

### Q31.

#### Explain memory leaks in Node.js — how to detect and fix them.

##### ■ *Node.js Internals & Debugging*

##### ◆ **Key Points:**

- Common causes: uncleaned event listeners, global variables, closures holding large data
  - Detection: `node --inspect` + Chrome DevTools Heap Snapshot
  - Use `clinic.js` or `0x` for profiling in production
  - Fix: always remove event listeners, avoid storing large objects globally
  - Monitor `process.memoryUsage()` in health endpoints
- 

### Q32.

#### How do you implement caching in a Node.js + MongoDB API?

##### ■ *Performance*

##### ◆ **Key Points:**

- In-memory (process-level): `node-cache` for single-instance apps
- Distributed: Redis for multi-instance/microservice setups
- Cache strategies: Cache-aside, Write-through, Write-back
- Invalidation: TTL-based, event-driven (clear on update/delete)
- Never cache user-specific sensitive data without scoping to `userId`

```
const redis = require('ioredis');
const client = new redis();
async function getUser(id) {
  const cached = await client.get(`user:${id}`);
  if (cached) return JSON.parse(cached);
  const user = await User.findById(id).lean();
  await client.setex(`user:${id}`, 3600, JSON.stringify(user));
  return user;
}
```

---

### Q33.

#### What is the Aggregation Pipeline \$lookup and how does it differ from SQL JOIN?

##### ■ MongoDB Aggregation

###### ◆ Key Points:

- \$lookup performs left outer join between two collections
- Embeds joined documents as array in parent document
- Less efficient than SQL JOINS for complex relationships
- Use \$unwind to flatten the resulting array
- Consider embedding data instead of \$lookup for read-heavy patterns

```
db.orders.aggregate([
  { $lookup: {
    from: 'users',
    localField: 'userId',
    foreignField: '_id',
    as: 'userInfo'
  }},
  { $unwind: '$userInfo' }
]);
```

---

### Q34.

#### How do you handle graceful shutdown in a Node.js server?

##### ■ Production & DevOps

###### ◆ Key Points:

- Listen for SIGTERM, SIGINT signals
- Stop accepting new connections, wait for active requests to finish
- Close DB connections, Redis clients, message queue connections
- Set a force-kill timeout (e.g. 10s) for stuck requests

```
process.on('SIGTERM', async () => {
  server.close(async () => {
    await mongoose.connection.close();
    process.exit(0);
  });
  setTimeout(() => process.exit(1), 10000);
});
```

---

### Q35.

#### Explain event-driven architecture using EventEmitter in Node.js.

##### ■ Node.js Core

###### ◆ Key Points:

- EventEmitter is Node's built-in pub/sub mechanism
  - Classes extend EventEmitter; emit named events with data
  - Listeners register with .on(), one-time with .once()
  - Decouples components: payment service emits 'payment.success', email service listens
  - Be careful of memory leaks from unremoved listeners
-

### Q36.

How do you design a MongoDB schema for a social media app (users, posts, follows, likes)?

#### ■ Schema Design

##### ◆ Key Points:

- Embed frequently co-read data (post with author subset, not full User)
  - Reference for many-to-many (follows: {followerId, followeeId} collection)
  - Likes: embed count on Post, separate Likes collection for who-liked queries
  - Denormalise for read-heavy patterns; accept some update complexity
  - Use compound indexes on (postId, userId) for like uniqueness
- 

### Q37.

What are the security headers every production Express app should send?

#### ■ Security

##### ◆ Key Points:

- Use helmet.js — sets 11+ security headers automatically
- X-Content-Type-Options: nosniff (prevent MIME sniffing)
- Strict-Transport-Security: enforce HTTPS
- Content-Security-Policy: restrict resource origins
- X-Frame-Options: DENY (prevent clickjacking)

```
const helmet = require('helmet');
app.use(helmet());
// Or fine-grained:
app.use(helmet.contentSecurityPolicy({ directives: { defaultSrc: ["'self'"] } }));
```

---

### Q38.

Explain the difference between horizontal and vertical scaling for a Node.js API.

#### ■ Scalability

##### ◆ Key Points:

- Vertical: bigger server (more RAM/CPU) — simple, has limits, single point of failure
  - Horizontal: more server instances behind load balancer — fault tolerant, unlimited scale
  - Node.js cluster module spawns workers per CPU core (single machine)
  - PM2 handles clustering + auto-restart in production
  - Stateless design required for horizontal scale — sessions in Redis, not memory
-

### Q39.

#### How would you debug a slow MongoDB query in production?

##### ■ MongoDB Debugging & Performance

###### ◆ Key Points:

- Enable slow query log: `db.setProfilingLevel(1, { slowms: 100 })`
  - Use `.explain('executionStats')` on the query
  - Look for COLLSCAN (full collection scan) — needs an index
  - Add compound index matching query fields + sort fields
  - Check index usage in Atlas Performance Advisor
- 

### Q40.

#### What is RBAC and how do you implement it in Express?

##### ■ Authorization

###### ◆ Key Points:

- Role-Based Access Control: users have roles, roles have permissions
- Store role in JWT payload or fetch from DB in auth middleware
- Create middleware factory: `authorize('admin', 'manager')`
- Check role before sensitive operations in controllers
- Consider ABAC (attribute-based) for fine-grained control

```
const authorize = (...roles) => (req, res, next) => {
  if (!roles.includes(req.user.role)) {
    return res.status(403).json({ message: 'Forbidden' });
  }
  next();
};

router.delete('/users/:id', auth, authorize('admin'), deleteUser);
```

---

### Q41.

#### How do you implement file uploads securely in Express?

##### ■ File Handling

###### ◆ Key Points:

- Use multer for multipart/form-data parsing
- Validate MIME type AND file extension (double validation)
- Set file size limits (`maxSize`)
- Store on cloud (S3/GCS) not local disk in production
- Generate UUID filenames to prevent path traversal

```
const upload = multer({
  storage: multer.memoryStorage(),
  limits: { fileSize: 5 * 1024 * 1024 }, // 5MB
  fileFilter: (req, file, cb) => {
    const allowed = ['image/jpeg', 'image/png', 'image/webp'];
    cb(null, allowed.includes(file.mimetype));
  }
});
```

---



## Q42.

### What is the N+1 query problem and how do you solve it in Mongoose?

#### ■ MongoDB Performance

##### ◆ Key Points:

- N+1: fetching 1 list then making N separate queries for related data
- Example: fetch 100 posts, then query user for each post = 101 queries
- Fix with .populate() in Mongoose (does 2 queries total)
- Or use aggregation \$lookup for complex cases
- Use DataLoader pattern in GraphQL contexts

```
// N+1 Problem (BAD):
const posts = await Post.find({});
for (const post of posts) {
  post.author = await User.findById(post.userId); // N queries!
}
// Fix:
const posts = await Post.find({}).populate('userId', 'name email');
```

---

---

## ■ CODING / PRACTICAL TASKS

---

Hands-on implementation challenges — bring your IDE or whiteboard.

### Q43.

#### Build a rate-limited login endpoint with JWT + bcrypt.

##### ■ Coding Task

##### ◆ Key Points:

- POST /api/auth/login accepts {email, password}
  - Hash comparison with bcrypt.compare()
  - Issue access token (15m) + refresh token (7d) on success
  - Return 401 on invalid credentials (same message to prevent enumeration)
  - Apply express-rate-limit: 10 requests / 15 min on this route
- 

### Q44.

#### Write an Express middleware that validates request body using Joi.

##### ■ Coding Task

##### ◆ Key Points:

- Create a factory function that accepts a Joi schema
- Validate req.body inside the middleware
- Call next() on success, next(validationError) on failure
- Use abortEarly: false to return all validation errors at once

```
const validate = (schema) => (req, res, next) => {
  const { error } = schema.validate(req.body, { abortEarly: false });
  if (error) {
    const errors = error.details.map(d => d.message);
    return res.status(400).json({ errors });
  }
  next();
};
```

---

### Q45.

Write a MongoDB aggregation to get the top 5 most active users by post count.

#### ■ MongoDB Task

##### ◆ Key Points:

- \$group by userId, count posts with \$sum: 1
- \$sort by count descending
- \$limit 5
- \$lookup to join with users collection for name/email

```
db.posts.aggregate([
  { $group: { _id: '$userId', postCount: { $sum: 1 } } },
  { $sort: { postCount: -1 } },
  { $limit: 5 },
  { $lookup: { from: 'users', localField: '_id',
    foreignField: '_id', as: 'user' } },
  { $unwind: '$user' },
  { $project: { 'user.name':1, 'user.email':1, postCount:1 } }
]);
```

---

### Q46.

Implement a Node.js utility to read a large CSV file using streams without loading it all into memory.

#### ■ Coding Task

##### ◆ Key Points:

- Use fs.createReadStream for the file
- Pipe through a line-by-line parser (readline or csv-parser)
- Process each row as it arrives
- Emit 'done' event with row count when stream ends

```
const fs = require('fs');
const readline = require('readline');
async function processCSV(filePath) {
  const rl = readline.createInterface({
    input: fs.createReadStream(filePath)
  });
  let count = 0;
  for await (const line of rl) {
    // process line
    count++;
  }
  return count;
}
```

---

### Q47.

#### Design and code a generic error class hierarchy for an Express API.

##### ■ Coding Task

##### ◆ Key Points:

- Base AppError extends Error with statusCode, isOperational fields
- Subclasses: NotFoundError (404), ValidationError (400), UnauthorizedError (401)
- isOperational flag distinguishes expected vs unexpected errors
- Global handler checks isOperational, logs unexpected ones differently

```
class AppError extends Error {
  constructor(message, statusCode) {
    super(message);
    this.statusCode = statusCode;
    this.isOperational = true;
  }
}

class NotFoundError extends AppError {
  constructor(msg = 'Resource not found') {
    super(msg, 404);
  }
}
```

---

### Q48.

#### Write a Mongoose schema for an e-commerce Product with validation.

##### ■ MongoDB / Mongoose Task

##### ◆ Key Points:

- Fields: name, description, price, category, stock, images array, ratings
  - Validate: price > 0, stock >= 0, name required & trimmed
  - Virtual field for discountedPrice
  - Pre-save hook to slugify the product name
  - Index on category and price for common filter queries
-

---

## ■■ SYSTEM DESIGN QUESTIONS

---

Architecture & design thinking — scalability, trade-offs, real-world systems.

### Q49.

**Design a scalable REST API for a food delivery app (users, restaurants, orders, delivery).**

#### ■ System Design

##### ◆ Key Points:

- Entities: User, Restaurant, MenuItem, Order, DeliveryAgent
  - Separate services: Auth, Menu, Order, Notification, Tracking
  - Order state machine: PLACED → CONFIRMED → PREPARING → OUT\_FOR\_DELIVERY → DELIVERED
  - WebSocket or SSE for real-time order tracking
  - MongoDB for orders (flexible schema), Redis for driver location, sessions
- 

### Q50.

**How would you design a URL shortener backend (like bit.ly)?**

#### ■ System Design

##### ◆ Key Points:

- POST /shorten: generate 6-char Base62 code, store {code, originalUrl, userId, clicks}
  - GET /:code: lookup code, increment click counter (async), 301/302 redirect
  - Use Redis cache for hot URLs (most clicked)
  - Collision handling: regenerate if code exists
  - Analytics: store click events with timestamp, user-agent, IP (geo-lookup)
- 

### Q51.

**Design the authentication system for a multi-tenant SaaS app.**

#### ■ System Design

##### ◆ Key Points:

- Each tenant has own subdomain and isolated data
  - JWT includes tenantId in payload; middleware scopes all DB queries
  - Separate user pools per tenant or shared pool with tenant isolation
  - Support SSO (SAML/OAuth) for enterprise tenants
  - Audit logs per tenant for compliance
-

## Q52.

**How would you implement a real-time notification system in Node.js?**

### ■ System Design

#### ◆ Key Points:

- WebSocket (Socket.io) for persistent connection per user
  - Server-Sent Events (SSE) for simpler one-way push
  - Redis Pub/Sub for broadcasting across multiple server instances
  - Store unread notifications in MongoDB, mark read on fetch
  - Queue heavy notifications (emails, SMS) to Bull/BullMQ
- 

## Q53.

**Design a file upload service that processes images and stores them on S3.**

### ■ System Design

#### ◆ Key Points:

- Client uploads to presigned S3 URL directly (bypass server bandwidth)
  - S3 event triggers Lambda (or webhook) on upload complete
  - Worker processes: resize, compress, generate thumbnails (Sharp.js)
  - Store metadata (original URL, thumb URLs, userId, size) in MongoDB
  - Return CDN URLs (CloudFront) to client, never direct S3 URLs
- 

## Q54.

**How would you implement an API Gateway pattern for microservices?**

### ■ System Design

#### ◆ Key Points:

- Single entry point routing requests to downstream services
  - Responsibilities: auth, rate limiting, SSL termination, request logging
  - Service discovery: hardcoded URLs (dev), Consul/K8s DNS (prod)
  - Circuit breaker pattern (opossum library) to handle service failures
  - Aggregate responses from multiple services in one gateway call
- 

## Q55.

**Design the database schema and indexing strategy for a chat application.**

### ■ System Design / Schema Design

#### ◆ Key Points:

- Collections: Users, Conversations, Messages
  - Message: {conversationId, senderId, content, type, readBy[], createdAt}
  - Compound index on (conversationId, createdAt) for message history
  - Index on readBy for unread count queries
  - Cap old messages with TTL index or archive to cold storage
  - Use change streams to push new messages via WebSocket
-